Question 1

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.ensemble import BaggingClassifier,RandomForestClassifier,AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# Load dataset
data = pd.read_csv('Dataset_Day12.csv')
print("Dataset Loaded Successfully.\n")
X = data.drop('Outcome', axis=1)
y = data['Outcome']

missing_cols = ['Glucose', 'BloodPressure', 'BMI', 'DiabetesPedigreeFunction']

# Replace 0s with median (excluding zeros)
for col in missing_cols:
    median_val = data[data[col] != 0][col].median()
    data[col] = data[col].replace(0, median_val)
print(data[missing_cols].describe())
print("Missing data was found in four columns. These were replaced with median values, " \
"which is robust to outliers and maintains the central tendency.")
```

```
          Glucose  BloodPressure        BMI  DiabetesPedigreeFunction

          Glucose  BloodPressure        BMI  DiabetesPedigreeFunction
          Glucose  BloodPressure        BMI  DiabetesPedigreeFunction
count  768.000000     768.000000  768.000000                768.000000
count  768.000000     768.000000  768.000000                768.000000
mean   121.656250      72.386719   32.455208                  0.471876
std     30.438286      12.096642    6.875177                  0.331329
min     44.000000      24.000000   18.200000                  0.078000
Missing data was found in four columns. These were replaced with median values, which is robust to outliers and maintains the central tendency.
Original dataset shape: (768, 7)
50%    117.000000      72.000000   32.300000                  0.372500
75%    140.250000      80.000000   36.600000                  0.626250
Missing data was found in four columns. These were replaced with median values, which is robust to outliers and maintains the central tendency.
```

Question 2

```python
def remove_outliers_iqr(dataframe, columns):
    df_clean = dataframe.copy()
    for col in columns:
        Q1 = df_clean[col].quantile(0.25)
        Q3 = df_clean[col].quantile(0.75)
        IQR = Q3 - Q1
        lower = Q1 - 1.5 * IQR
        upper = Q3 + 1.5 * IQR
        df_clean = df_clean[(df_clean[col] >= lower) & (df_clean[col] <= upper)]
    return df_clean

numeric_cols = data.columns.drop('Outcome')
df_clean = remove_outliers_iqr(data, numeric_cols)

# Show number of rows before and after outlier removal
print(f"Original dataset shape: {data.shape}")
print(f"Dataset shape after outlier removal: {df_clean.shape}")
print(f"Number of rows removed as outliers: {data.shape[0] - df_clean.shape[0]}")
print("Outliers can skew distance calculations in kNN. Removing them ensures better model stability and less variance in prediction.")
```

```
Original dataset shape: (768, 7)
Dataset shape after outlier removal: (698, 7)
Number of rows removed as outliers: 70
Outliers can skew distance calculations in kNN. Removing them ensures better model stability and less variance in prediction.
Data split into 80% training and 20% testing sets.
```

Question 3

```python
import pandas as pd
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
from sklearn.naive_bayes import GaussianNB
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# Load the dataset
df = pd.read_csv("Dataset_Day12.csv")

#Separate features and target
X = df.drop("Outcome", axis=1)
y = df["Outcome"]

# Split the data (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# a. Naive Bayes Classifier

nb = GaussianNB()
nb.fit(X_train, y_train)
y_pred = nb.predict(X_test)

print("\n Naive Bayes (80-20 Split)")
print(f"Accuracy:  {accuracy_score(y_test, y_pred):.3f}")
print(f"Precision: {precision_score(y_test, y_pred):.3f}")
print(f"Recall:    {recall_score(y_test, y_pred):.3f}")
print(f"F1 Score:  {f1_score(y_test, y_pred):.3f}")

# b. 10-Fold Cross Validation
cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)

print("\n Naive Bayes (10-Fold Cross Validation)")
print(f"CV Accuracy:  {cross_val_score(nb, X, y, cv=cv, scoring='accuracy').mean():.3f}")
print(f"CV Precision: {cross_val_score(nb, X, y, cv=cv, scoring='precision').mean():.3f}")
print(f"CV Recall:    {cross_val_score(nb, X, y, cv=cv, scoring='recall').mean():.3f}")
print(f"CV F1 Score:  {cross_val_score(nb, X, y, cv=cv, scoring='f1').mean():.3f}")
```

```
 Naive Bayes (80-20 Split)
Accuracy:  0.753
Precision: 0.644
Recall:    0.691
F1 Score:  0.667

 Naive Bayes (10-Fold Cross Validation)
CV Accuracy:  0.755
CV Precision: 0.673
CV Recall:    0.608
CV F1 Score:  0.635
```